Ranking Web Pages for Content Refresh Using Search Performance Data

Saad Wasay Chauhdary
FlyRank Machine Learning Internship
Institute of Space Technology, Islamabad, Pakistan
saadwasay813@gmail.com

***Abstract*—Content teams that manage many web pages need a practical way to decide which pages should be refreshed first. This paper studies whether search performance data can support this decision, and whether machine learning gives a real advantage over a simple, easy to explain ranking rule. A label was built from the change in average search position between two time periods, using only features known before the second period began, so a model could not see the outcome in advance. A rank based heuristic was compared against a decision tree, a random forest, and logistic regression, using client grouped cross validation on two separate datasets, a small starter sample and a large real search warehouse. Across three separate comparisons, no model beat the simple heuristic by a statistically meaningful margin, and the heuristic reached 92.5 percent precision among its top ranked pages on the full warehouse data, which supports using the simpler and more transparent method in practice. Standard threshold-based classification metrics – accuracy, precision, recall, F1, and confusion matrices – were also computed on the warehouse tests as a supplementary check, and confirm the same conclusion: no method separates itself meaningfully from the baseline once the ranking-level result is already known.**

***Index Terms*—content refresh, search performance, ranking, feature leakage, cross validation, decision support**

I. Introduction

A content team that manages thousands of web pages cannot refresh all of them at once. Editorial time is limited, so some way of ranking pages by refresh priority is needed. The question this paper asks is narrow on purpose: given a page's observed search performance up to a certain point in time, can that page be ranked well enough to be useful for prioritization, and is a complex model actually needed to do this well?

This work does not try to explain or reverse engineer Google's ranking algorithm. It also does not claim that refreshing a page causes any particular change in its performance. The goal is decision support: given the data a content team already has, produce a ranked list of pages with a stated reason for each one, and be honest about where that list is likely to be wrong.

A second question runs through the whole study. Machine learning is often assumed to be the right tool for this kind of ranking problem. Rather than assume that, this paper treats it as something to test. A simple heuristic was built first, and every model tried afterward had to prove it did better than that heuristic on the same data, using the same validation method.

This paper was completed as part of the FlyRank Machine Learning Internship research track, undertaken independently of the author's coursework at Institute of Space Technology.

II. Related Work

Tree based methods are a common choice for tabular business data because they handle non linear relationships without much preprocessing. The classification and regression tree approach used here follows the general method described by Breiman et al. [1], and the random forest model follows Breiman's later ensemble method [2]. All models were implemented using the scikit-learn library [3]. The data used in this study was queried directly from Parquet files using DuckDB [4], which made it possible to work with a warehouse of close to 80 million rows without downloading the full dataset.

III. Data

Two datasets were used. The first was a small anonymized sample of about 30,000 web pages, used to develop and test the overall method cheaply before touching the full data. The second was the full FlyRank ML Internship Warehouse, released as a gated, anonymized dataset on Hugging Face for this internship. All identifiers in both datasets are hashed and do not reveal any real client, domain, or query.

The warehouse data used here comes from the fact_content_daily_performance table, restricted to March 2026, a single mid range month chosen deliberately instead of the most recent month in the release. The most recent month is reserved as a sealed test period for this internship track and should not be used to develop labels or features.

This table records one row per page per client per day. This grain was checked directly rather than assumed, by grouping on those three fields and confirming there were no duplicate rows, across all 9,841,378 rows in the chosen month.

Not every row has usable search performance data. Only 36.7 percent of rows had search console data available. This was checked at both the client level and the day level. Availability was stable across the days in the month, which rules out a rollout timing effect, but a number of clients showed zero percent availability across very large row counts, which points to a structural cause: some clients in this dataset simply never connected their search console. Rows and pages from those clients are excluded from this study, since there is no search signal to judge decline from.

The month was split into two halves. The first half was used to build features. The second half was used to define the outcome. Only pages with real search data present in both halves were kept, since a page missing from either half cannot be given an honest label. This left 141,467 pages across 43 clients. The page count per client is very uneven, ranging from 2 to 25,437 pages for a single client, which is treated as a limitation throughout this paper rather than something to hide.

A second table, dim_content, holds page level attributes such as word count, content type, search volume, and the date the page was created. This table was joined onto the modeling population after checking that it held one row per page, with no risk of duplicating rows through the join.

IV. Methodology

A. Label Definition

There is no column in this data that states whether a page needs a refresh. A proxy label had to be built. The chosen label was based on average search ranking position, comparing the first half of the month to the second half. If a page's average position got worse in the second half, it was labeled as needing a refresh.

Click through rate was tried first as the basis for this label, but it turned out to be zero for around 63 to 64 percent of pages in both halves of the month. A label built on a value that sits at zero most of the time is not a useful label, so position was used instead, which has a much wider and more even spread of values.

B. Feature Engineering and Leakage Prevention

Every feature used to predict the label was restricted to information available from the first half of the month only, before the outcome period even began. Five base features were used: average impressions, average clicks, click through rate, average position, and the number of days in the first half with any recorded impressions.

Three additional features were later added from the page attributes table: word count, character count, and search volume. Two page level date fields were tested and then excluded, once it became clear they described the state of the page as of the data export date rather than as of the middle of the month being studied. Checking this directly showed that every single non missing value in one of those fields fell after the cutoff date used to define the label, which means that field would have described the outcome rather than predicted it. A page level backlink count was excluded for the same reason, since only a single current snapshot of it was available, not a snapshot from the correct point in time.

This kind of check was treated as mandatory for every new feature, not just the ones that looked suspicious at first glance. A separate check, aimed specifically at demonstrating this risk rather than avoiding it, deliberately added the second half outcome value as a feature. Doing this moved the model's score from an honest 0.6412 to a perfect 1.0000, confirming that the validation process used in this study is sensitive enough to catch this kind of mistake if it happened by accident.

A related but quieter version of the same problem was found earlier, while working with the smaller starter sample. A missing value indicator on one feature turned out to carry most of the weight in an early decision tree. On inspection, that indicator was almost perfectly aligned with pages that had no prior history at all, which are pages that cannot be labeled as declining under this definition simply because they have nothing to decline from. Those pages, along with pages with no search visibility in either period, were removed from that dataset before repeating the analysis, since asking whether a brand new page needs a refresh is not a meaningful question.

C. Baseline Construction

Before any model was trained, a simple heuristic was built using only feature strength measured on the training data, so that no part of the baseline was tuned using the same data it was later scored on. Feature values were converted into their percentile rank within the training set rather than scaled by minimum and maximum, since an early version of this baseline used minimum and maximum scaling and was found to be dominated by a single feature with extreme outlier values, purely because of how that scaling method behaves. Percentile ranking avoided that problem.

On the warehouse data, only average position carried a clear signal strong enough to be worth keeping in the heuristic. A second candidate feature, based on the count of active days, was tested in combination with position, but comparing the single feature version against the combined version directly showed the combined version was less reliable in the range of results that would realistically be acted on. The single feature heuristic was kept as the final baseline for that reason.

D. Validation Strategy

Pages from the same client are likely to share client level patterns, so a plain random split would let information leak between training and test data through shared client behavior. To avoid this, every split in this study grouped all of a client's pages together, so a client's pages were never split between training and test at the same time.

Five fold cross validation was used, grouped by client, so that every client's pages were used for testing exactly once. Because the number of clients is small and very uneven in size, several candidate random seeds were compared before committing to one, based only on how balanced the resulting split was, decided before any model was trained on that split.

E. Classification Metrics at a Fixed Threshold

AUC and Precision@K are threshold-free measures of ranking quality, which is the primary concern of this study. As a supplementary check, standard classification metrics – accuracy, precision, recall, and F1 – were also computed at a fixed decision threshold of 0.5, applied identically to every method, including the heuristic baseline, whose percentile-ranked score is already bounded to $[0, 1]$. Predictions were pooled out-of-fold across all five cross validation folds before computing each metric, so that every page contributes exactly

one prediction, from whichever fold held it out. This was done for both warehouse feature sets; it was not repeated on the smaller starter sample, since this check was added later in the study specifically to validate the warehouse-scale conclusions in a second, standard format.

V. Results

TABLE I
MEAN CROSS VALIDATED AUC BY METHOD AND DATASET

Method	Starter sample	Warehouse (5 feat.)	Warehouse (8 feat.)
Heuristic baseline	0.5699	0.6373	0.6361
Decision tree	0.5752	0.6313	0.6248
Random forest	0.5870	0.6379	0.6362
Logistic regression	0.5542	0.6230	0.6131

TABLE II
PAIRED SIGNIFICANCE AGAINST THE BASELINE, BY FOLD

Method	Starter sample p	Wareh. 5 feat. p	Wareh. 8 feat. p
Decision tree	0.809	0.265	0.075
Random forest	0.401	0.892	0.985
Logistic regression	0.442	0.039	0.055

Table I shows the mean cross validated AUC for each method, on both datasets, with and without the added content attributes on the warehouse data. Table II shows the result of a paired significance test comparing each model against the heuristic baseline, fold by fold, using the standard threshold of 0.05. Fig. 1 shows the same comparison as a chart, with a dotted line marking the random chance level of 0.5.

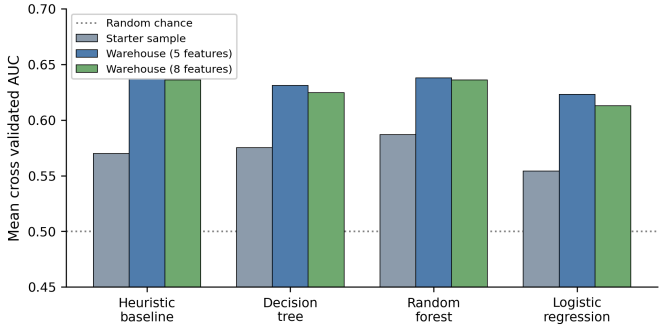

Fig. 1. Mean cross validated AUC by method, across the starter sample and the two warehouse feature sets.

Across all three comparisons, no model beat the baseline by a significant margin. The random forest came closest each time, but never with a p value below 0.4. Logistic regression was the only method that fell significantly below the baseline, and only in the five feature warehouse test, which suggests the relationship between these features and the label is not well described by a straight line.

Adding the three content attributes to the warehouse features did not change this picture. The random forest mean AUC changed from 0.6379 to 0.6362, a difference far smaller than the spread seen across folds, and still not significant against the baseline.

On the full warehouse population, the chosen baseline reached 92.5 percent precision among its top 200 ranked pages, against an overall rate of 54.1 percent of pages needing a refresh under this label. This is the number most directly relevant to how the ranking would actually be used, since a content team is realistically working through a list of a few dozen to a few hundred pages at a time, not the whole population at once.

A. Threshold-Based Classification Metrics

The metrics above are threshold-free, and remain the primary evidence in this study. For completeness, and to express the result in a second, standard format, accuracy, precision, recall, and F1 were also computed at a fixed threshold of 0.5, pooled out-of-fold, for the two warehouse tests. Table III reports these for the five-feature test; Table IV reports the same for the eight-feature test with content attributes added.

TABLE III
CLASSIFICATION METRICS @ THRESHOLD 0.5, WAREHOUSE (5 FEATURES), POOLED OUT-OF-FOLD

Method	Acc.	Prec.	Rec.	F1
Heuristic baseline	0.608	0.649	0.598	0.622
Decision tree	0.607	0.634	0.648	0.641
Random forest	0.607	0.647	0.601	0.624
Logistic regression	0.604	0.607	0.762	0.676

TABLE IV
CLASSIFICATION METRICS @ THRESHOLD 0.5, WAREHOUSE (8 FEATURES), POOLED OUT-OF-FOLD

Method	Acc.	Prec.	Rec.	F1
Heuristic baseline	0.608	0.650	0.598	0.623
Decision tree	0.603	0.637	0.617	0.627
Random forest	0.612	0.652	0.605	0.628
Logistic regression	0.597	0.599	0.772	0.674

TABLE V
CONFUSION MATRICES, WAREHOUSE (5 FEATURES), POOLED OUT-OF-FOLD (COUNTS OF PAGES)

Method	TN	FP	FN	TP
Heuristic baseline	40,284	24,699	30,788	45,696
Decision tree	36,326	28,657	26,900	49,584
Random forest	39,935	25,048	30,496	45,988
Logistic regression	27,216	37,767	18,202	58,282

All four methods land within a narrow band of accuracy, between 0.597 and 0.612, which is consistent with the AUC values in Table I: an AUC in the 0.62–0.64 range does not translate into a threshold-based split that looks dramatically different across methods. Because none of the methods discriminate strongly, these threshold-based numbers add little beyond what AUC and Precision@K already establish, and

they are reported here as a supplementary, standard-format view rather than as the primary evidence for this study's conclusion.

The one distinct pattern is in logistic regression, which reaches the highest recall of any method (0.762 on the five-feature test, 0.772 with content attributes added) at the cost of the lowest precision (0.607 and 0.599, respectively) and roughly 50 percent more false positives than any other method, as shown in Table V. This gives a concrete, numeric face to the AUC finding already reported: logistic regression is the only method that falls significantly below the baseline, and only on the five-feature test, which was interpreted as evidence that the relationship between these features and the label is not well described by a straight line. A linear decision boundary at a fixed threshold, combined with class-weight balancing, pushes logistic regression to label far more pages as needing refresh than the data actually supports, which produces exactly this precision and recall imbalance.

VI. Error Analysis

Looking only at overall accuracy hides where a ranking method actually fails, so the top ranked pages and the missed pages were examined directly rather than just reported as a single score.

Among the pages that were true decliners but ranked lowest by the model, the typical page had 9 days of recorded activity in the first half of the month, compared to only 2 days for pages correctly identified as not declining. In other words, pages with a longer and more consistent history of visibility are the ones most likely to be missed when they start to decline. This makes sense given how the baseline is built, since a page with a long steady history looks stable by the measures used here, right up until it is not.

This was treated as a limitation to disclose rather than a problem to quietly fix, since fixing it without more data risks trading one kind of error for another without evidence that the trade is worth it.

VII. Ranked Recommendations

The final output ranks every page by the percentile of its baseline score and groups pages into three tiers. The top 10 percent are marked for immediate refresh, the next 20 percent are marked for monitoring, and the remaining 70 percent require no action for now. On the full population this produced 14,147 pages marked for immediate refresh, 28,283 marked for monitoring, and 99,037 marked as needing no action.

Each ranked page also carries a short, plain language reason rather than only a number, for example noting that a page is visible in search results but is not earning clicks, or that its visibility has been inconsistent. Pages that match the blind spot found in the error analysis, meaning pages with a long steady history of visibility, are flagged separately for manual review regardless of their computed tier, since the model is known to under detect decline in that group.

VIII. Limitations

Several limitations should be kept in mind when reading these results. Only pages from clients with search console data connected are covered, which is roughly a third of the pages in the warehouse for the month studied. The label used here describes a direction of change in ranking position, not its size or its business importance, and it reflects an association observed in one month of data rather than a cause and effect claim. The client panel is small and uneven, with as few as two pages for some clients and tens of thousands for others, so results from any single cross validation split should be read as directional rather than exact.

One finding is worth naming directly because it did not hold up between datasets. Content age was the strongest single signal in the smaller starter sample, but carried almost no signal in the full warehouse data. This is reported honestly rather than explained away, since it may reflect a difference in how the two labels were built rather than a real change in the underlying pattern, and this kind of disagreement between datasets is exactly the sort of thing a single dataset study would never surface.

The threshold-based classification metrics in Section V-A carry an additional limitation of their own: they were computed only for the two warehouse tests, at a single fixed threshold chosen for consistency rather than tuned for any particular use case, and should be read alongside the AUC and Precision@K results rather than in place of them.

IX. Conclusion

Across two datasets, three separate evaluations, and three model families, no model produced a statistically meaningful improvement over a simple, fully transparent ranking heuristic based on prior search position. This is treated here as a real and useful finding rather than a disappointing one. It means a content team can use a method that is easy to explain and easy to audit, without giving up measurable performance for the sake of using a more complex model. The heuristic reached 92.5 percent precision among its top ranked pages on the full warehouse population, which is strong enough to support real prioritization decisions, while the known blind spot around long established pages is disclosed directly in the output rather than hidden behind a single score. Standard classification metrics, computed as a supplementary check at a fixed threshold, tell the same story in a second format: no method separates itself meaningfully from the baseline, with the partial exception of logistic regression, whose recall/precision imbalance reinforces, rather than contradicts, the paper's central conclusion.

X. Reproducibility

All code used in this paper is organized as executed notebooks in the accompanying repository, under work/notebooks/. This includes the starter sample exploration and full model comparison notebook, the warehouse data contract notebook that verifies table grain and demonstrates the deliberate leakage check described in Section IV-B, and every earlier weekly

assignment notebook for this internship track. Each notebook runs top to bottom without errors and its outputs are saved directly in the file, so results can be checked without re running any query against the gated warehouse. The repository is available at https://github.com/saadwasay210/saad-flyrank-ml-internship.

ACKNOWLEDGMENT

Built on the FlyRank ML Internship dataset, https://flyrank.ai.

REFERENCES

[1] L. Breiman, J. Friedman, R. Olshen, and C. Stone, Classification and Regression Trees. Wadsworth, 1984.

[2] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[4] M. Raasveldt and H. Muhleisen, "DuckDB: an embeddable analytical database," in Proc. ACM SIGMOD Int. Conf. Management of Data, 2019, pp. 1981–1984.